

NAO Navigator

1st Richard Fabian Willke

Electrical Engineering and Information Technology
Technical University Munich
Munich, Germany
go34qil@mytum.de

2nd Rohan Singh

Robotics, Cognition and Intelligence
Technical University Munich
Munich, Germany
rohan.singh@tum.de

3rd Yueyang Zhang

Electrical Engineering and Information Technology
Technical University Munich
Munich, Germany
yueyang.zhang@tum.de

Abstract—Navigation and exploration are one of the cornerstones for mobile robotics. They present an existential ability for the robot to not only get from a starting position to a desired goal position but to also achieve this in a save and coordinated manner. In this paper we introduce an approach on navigation and exploration though a partially known environment on a NAO robot. The goal of this project is to design an algorithm implemented on a NAO robot to enable the robot to explore an environment while steering to its desired goal.

Index Terms—Humanoid, Robotics, Path Planning, Exploration

I. INTRODUCTION

A mobile robot is defined as a robot that is able to change its position within an environment. Ensuring a secure and reliable navigation through that environment is one of the most crucial requirements for this to be achieved. A navigation system needs to be capable of two things. First of all, it needs to correctly estimate the position of the robot in the real world. Secondly, it needs to find a trajectory that the robot should be able to follow to reach the desired goal.

The first requirement is achieved through the choice of an appropriate state estimator. Throughout the last decades, several different approaches have been tried out, each with their own strengths and weaknesses. One notable approach that is still being used to this day is the Kalman Filter (KF). The first KF has been used for the moon-landing operation in 1969. Since then, it has been advanced and adapted to this day for various state of the art systems.

The second requirement for the navigation algorithm requires a path planning algorithm. A path planning problem is a variant of search algorithm problem. These problems are characterized by the use of search trees through which the algorithm searches to find the shortest path to the desired goal. To name a few, popular approaches include the A* Algorithm, Dykstra's Algorithm and Rapidly-exploring Random Tree (RRT). All of these algorithms rely on a search through a state tree. To achieve this, the world map model has to be discretized first, often through voxelization into a grid world. The search algorithm is set to explore the tree associated with the discretized map.

Each subsequent state of the tree represents a different position on the map. To find the most optimal path, a weight is set for each transition of one state to a subsequent state. The goal of the algorithm is to find a path through the tree that minimizes the total cost to reach the goal.

The following chapters give further insight into the posed problem and solution approach. The paper concludes with a summary and discussion on the results from system performance tests.

II. PROBLEM FORMULATION

In this project, NAO is tasked with navigating a challenging environment while walking towards a human. Intermittent tracking loss makes self-localization difficult, so the robot should keep track of the last known landmarks and obstacles so that it can plan a path and follow it robustly towards a human at the edge of the terrain. To achieve this, the robot needs to be able to locate itself within the environment consisting of the goal and obstacles and subsequently plan its trajectory accordingly.

III. METHODOLOGY

To achieve the goals of this task, the robot first of all needs a method for distinguishing and locating obstacles and the goal. To do so, we rely on Aruco markers. They store the information about each object's position and orientation in relation to the robot's respective camera frames. The information is subsequently used to plan the trajectory the robot will have to follow to reach the goal. For this, the A* algorithm is utilized. The trajectory consists of linear segments, which are smoothed using Bezier curves. These curves lay the groundwork for planning the footsteps that the NAO robot will have to take to reach the desired goal. An optimal controller is used to calculate the trajectory. To estimate the robot position and orientation at any given time during the following of the trajectory, a Kalman Filter is introduced.

The project code is accessible via the referenced gitlab¹. The

¹https://gitlab.lrz.de/00000000014AEBD4/hrs_ws/-/tree/final_submission?ref_type=heads

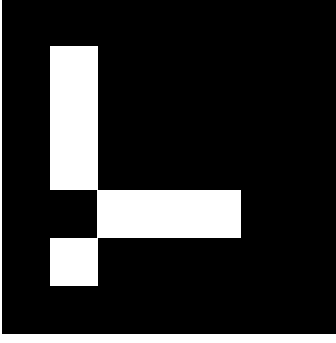


Fig. 1: Example of an aruco marker representing the ID 12. This marker was created using [5]

following sections go into further detail into the functionality of the individual modules. For documentation on the project code, please refer to the documentation of the README and within the code on the repository's website.

A. ArUco based Exploration

ArUco markers are part of the OpenCV library [2]. The module is based on the ArUco library. An ArUco marker is a synthetic square marker composed by a wide black border and a inner binary matrix which determines its identifier. The binary matrix consists of a set of white and black tiles in a 2D grid form. An example is shown in figure 1.

The ArUco marker holds the information about its own position and orientation relative to the camera's optical frame. This information lays the basis for easily discovering objects within a given map.

The updating of the map relative to the world frame happens every time a new obstacle is discovered or if the goal has moved significantly enough to justify a replanning of the A* trajectory.

The robot uses both of its cameras to detect and estimate the pose of ArUco markers in the world, relative to the optical frame of each camera. The positions and orientations of the detected objects in the optical frames are each transformed to the world coordinate frame, allowing for precise updates of the environment. A dictionary of previously discovered obstacles is created, adding further robustness to the system to noises in the detection. This is also particularly important in cases where the executed trajectory deviates from the planned path, for example when the robot falls over. The robot would have to rediscover at least one of the known obstacles to correctly estimate its own position in the world again.

Two separate callback functions are created, each utilizing their camera. The top camera is used to detect the ArUco marker associated with the goal. The bottom camera is used to detect the obstacles in close vicinity of the robot. This serves as an initial plausibility check. Because the ArUco marker representing the goal is on a significantly higher altitude compared to the obstacles, it only makes sense for the robot to identify the goal ArUco via the top camera and the obstacles through the bottom camera.

B. Kalman Filter for State Estimation

The localization stack is one of the most critical perspectives for a successful navigation task. The IMU (Inertial measurement unit) readings are normally noised and hence are suboptimal for localization. To secure high quality of estimation of robot's position, a Kalman filter [7] is integrated into the system to obtain an optimal estimation. Considering the limited time of the project, we essentially implemented a linear Kalman filter to acquire the X-Y values of the robot's location w.r.t. odometer origin.

Theoretically, Kalman filter can be divided into 2 working phases: prediction and update. According to the previous observations on the system, the algorithm keeps computing the corresponding error covariance and Kalman gain, the former one evaluates the error in system's state transition while the latter weights the prediction and observation according to the covariance in order to give an accurate state estimation in noised condition.

For the project's needs, we assume noises to be invariant with time and set the system's state as the concatenation of robot's position and velocity, i.e., the noised readings of odometer and IMU, meanwhile, the acceleration of the NAO robot is regarded as system's input. The noised system model can be interpreted as follows:

$$\begin{aligned} \mathbf{X}_k &= \mathbf{A}\mathbf{X}_{k-1} + \mathbf{B}\mathbf{u}_k + \mathbf{W}, \\ \begin{bmatrix} x_k \\ y_k \\ \dot{x}_k \\ \dot{y}_k \end{bmatrix} &= \begin{bmatrix} \mathbf{I}_{2 \times 2} & \Delta t \mathbf{I}_{2 \times 2} \\ \mathbf{0}_{2 \times 2} & \mathbf{I}_{2 \times 2} \end{bmatrix} \begin{bmatrix} x_{k-1} \\ y_{k-1} \\ \dot{x}_{k-1} \\ \dot{y}_{k-1} \end{bmatrix} \\ &\quad + \begin{bmatrix} \frac{1}{2} \Delta t^2 \mathbf{I}_{2 \times 2} \\ \Delta t \mathbf{I}_{2 \times 2} \end{bmatrix} \begin{bmatrix} \ddot{x}_k \\ \ddot{y}_k \end{bmatrix} + \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ w_4 \end{bmatrix}. \end{aligned} \quad (1)$$

As the position of robot being observed variable, the noised observation model is constructed as

$$\begin{aligned} \mathbf{Z}_k &= \mathbf{H}\mathbf{X}_k + \mathbf{V}, \\ \begin{bmatrix} x_k \\ y_k \end{bmatrix} &= \begin{bmatrix} \mathbf{I}_{2 \times 2} & \mathbf{0}_{2 \times 2} \end{bmatrix} \begin{bmatrix} x_k \\ y_k \\ \dot{x}_k \\ \dot{y}_k \end{bmatrix} + \begin{bmatrix} z_1 \\ z_2 \end{bmatrix}. \end{aligned} \quad (2)$$

For that the noises in both models (equation 1 and 2) are unavailable, the prediction for the next system's state based on its current state $\hat{\mathbf{X}}_{k|k-1}$ is given by

$$\hat{\mathbf{X}}_{k|k-1} = \mathbf{A}\hat{\mathbf{X}}_{k-1|k-1}, \quad (3)$$

where $\hat{\mathbf{X}}_{k-1|k-1}$ stands for corrected previous state.

Given an initial guess of covariance, it is updated with

$$\mathbf{P}_{k|k-1} = \mathbf{A}\mathbf{P}_{k-1|k-1}\mathbf{A}^\top + \mathbf{W},$$

and therefore Kalman gain is obtained by

$$\mathbf{K}_k = \mathbf{P}_{k|k-1}\mathbf{H}^\top(\mathbf{H}\mathbf{P}_{k|k-1}\mathbf{H}^\top + \mathbf{V})^{-1},$$

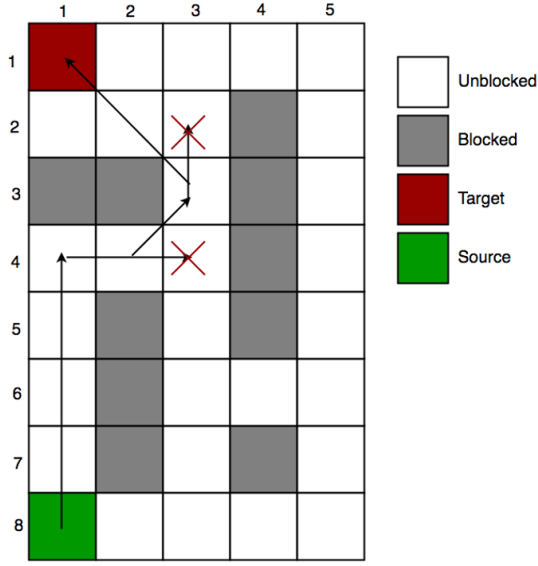


Fig. 2: Example of discretized map including obstacles, target and estimated optimal path. Original graphic by [3]

subsequently, we correct the current state as

$$\hat{\mathbf{X}}_{k|k} = \hat{\mathbf{X}}_{k|k-1} + \mathbf{K}_k(\mathbf{Z}_k - \mathbf{H}\hat{\mathbf{X}}_{k|k-1}).$$

The corrected state can be utilized as an accurate NAO position.

C. A* Path Planing

The A* algorithm is a form of graph search algorithm, used to find the shortest path to a goal as quickly as possible [4]. In the use-case at hand, this graph is an abstraction of the discretized map. The discretized map consists of square tiles as depicted in figure 2.

This discretized map consists of tiles, each with their respective neighboring tiles. For this example, a specific world state is described through the position of the robot on one specific tile. The robot can decide to go into any of the neighboring tiles, except for tiles outside of the map or tiles that are occupied by an obstacle. The transition of one state to another is executed via an action. This dynamic can be modeled by a state transition tree, where every node represents a state and every connection between states represents an action that leads to a state transition into a neighboring state. An illustration of this concept is depicted in figure 3. Note, that the constructed tree excludes recursions. This is important for search to eliminate any unnecessary maneuvers and to prevent the algorithm to get stuck in a loop of consecutive states. Thus, any states that have already been explored by the algorithm will be omitted from being visited again during the search to prevent these undesired behaviors.

A* is a search algorithm that utilizes heuristics to find the shortest path to the goal with as little exploration as possible. The A* algorithm uses a metric to weigh the priority of specific states. This enables the A* to determine which

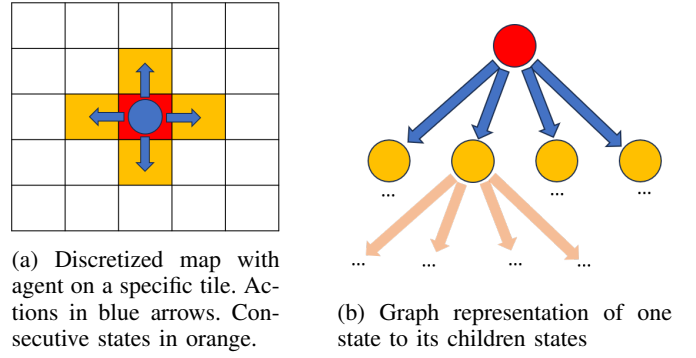


Fig. 3: Abstraction of map before it is processed by A*

states shall be explored in the future, making the exploration of the map significantly faster and computationally efficient compared to its predecessors such as depth first search or breadth search. The metric with which the A* operates consists of the backward cost $g(n)$ and the forward cost $h(n)$. The latter is often also referred to as the heuristic of the function. The total cost of a state is determined by the sum of the backward and forward cost.

$$f(n) = h(n) + g(n) \quad (4)$$

The backward cost is the true cost that has been observed when observed. In this project, it is equal to the amount of tiles that have been passed to reach from the starting position to the position of the tile. This distance is also referred to as the manhattan distance. For the A* to work as intended, an admissible heuristic needs to be used. An admissible heuristic is any heuristic $h(n)$, where it holds, that:

$$0 \leq h(n) \leq h^*(n) \quad (5)$$

Where $h^*(n)$ is the true minimal cost to the nearest goal. The heuristic used in this project is the euclidean distance between a given tile to the goal position. This is an admissible heuristic, since the euclidean distance is a strictly positive metric, including zero, that is always smaller or equal to the manhattan distance. For further proof, please refer to the Pythagorean theorem:

$$d^2 = \Delta x^2 + \Delta y^2 \geq 0 \quad (6)$$

Where the euclidean distance d always:

$$0 \leq d \leq |\Delta x| + |\Delta y| \quad (7)$$

Substituting $h(n) = d$ and the minimum cost $h^*(n) = \Delta x + \Delta y$ results in the desired result:

$$0 \leq h(n) \leq h^*(n) \quad (8)$$

D. Curve-Fitting with Bézier Curve

The A* generated trajectory is a set of a sequence of discrete points forming a curve composed by straight lines and sharp corners. Instead of directly compute foot steps based on the points, it is necessary to fit a suitable curve on the principle trajectory to ensure a natural walking behavior of the robot.

The classical approach to fit points into curves is polynomial curve fitting [6]. For our application, the fitting task is bound to be under-parameterized, meaning the number of given points is always larger than the order of polynomial.

For such task, after the order of polynomial n is defined, this algorithm recursively finds the coefficients for a polynomial that minimizes the distance of the curve and given points. E.g., given $n = 3$, the polynomial curve fitting returns a, b and c of

$$y = ax^2 + bx^1 + cx^0 \Rightarrow y = ax^2 + bx + c.$$

Under the scenario-setting of this project, the robot is required to walk along the generated curve, for which, the polynomial curve fitting has a fatal drawback: user has no control on the orientation at initial point. If the robot is required to take a step with a sharp turning, it will, with significant probability, fall down. To balance the robot, the θ between the subsequent steps should be constrained in a reasonable range.

To obtain a smooth curve especially at the initials, Bézier curve fitting [1] is implemented. The Bézier curves are parametric curves that are calculated by a set of points (or control points). The general formula of Bézier curve is given by

$$B_n(x) = \sum_{v=0}^n \beta_v b_{v,n}(x),$$

$$b_{v,n}(x) = \binom{n}{v} x^v (1-x)^{n-v},$$

with $b_{v,n}(x)$ defined as Bernstein polynomial, n and β_v are Bernstein order and its coefficient, respectively. To simplify the process, Bézier curve can be approximated by:

$$B_n(x) = \sum_{v=0}^n f\left(\frac{v}{n}\right) b_{v,n}(x) = \sum_{v=0}^n t b_{v,n}(x),$$

where $t \in [0, 1]$.

For intuition, imagine 3 points are given (P_0, P_1 and P_2) to fit a quadratic Bézier curve, meanwhile 2 of which are used to fit a linear Bézier curve. Thus we have

$$\text{linear: } (1-t)P_0 + tP_1, \quad (9)$$

$$\text{quadratic: } (1-t)^2P_0 + 2(1-t)tP_1 + t^2P_2. \quad (10)$$

The quadratic curve (equation 10) can be further converted into

$$(1-t)[(1-t)P_0 + tP_1] + t[(1-t)P_1 + tP_2], \quad (11)$$

with $[(1-t)P_0 + tP_1]$ and $[(1-t)P_1 + tP_2]$ are linear fittings from P_0 to P_1 and from P_1 to P_2 , respectively. One notices that the quadratic fitting is actually a fitting that is controlled

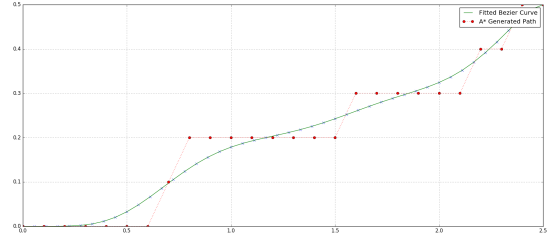


Fig. 4: Bézier curve fitting. Given path generated by A* algorithm are shown with red points, the fitting result is interpreted by green curve, blue crosses stand for principle steps (steps that aligned on the Bézier curve).

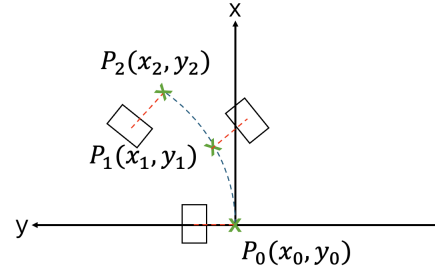


Fig. 5: An abstract logic for footstep planning. Subsequent principle steps in torso frame are interpreted by green crosses. Blue dashed curve stands for Bézier curve. The red dashed lines show the separation of feet.

by linear fittings through its control points. By setting $t = 1$, we adopt the full length of the fitted curve. For example, given 4 points, the following trajectory is generated (figure 4), which forms a better shape for NAO robot's execution.

In practice, the Bézier curve fitting is designed to utilize discrete points from A* algorithm as input, and its unit is converted in meters to suit the real map of the environment. After a curve is fitted, it is again discretized into points, the coordinates of them represent the positions of NAO's torso at each footstep w.r.t. its current torso frame.

E. Optimal Footstep Planning

The Bézier curve is given in NAO's torso frame, however, the robot takes the command for the next step w.r.t. the current supporting foot. Meaning that the fitted curve should firstly be converted into step plan and then transformed in corresponding frames.

As described in section III-D, the output of Bézier curve fitting is a set of torso positions at each footstep w.r.t. the robot's current torso frame. For simplicity, these positions are named as principle steps. 3 subsequent principle steps including initial one are interpreted as green crosses in figure 5.

The orientation at each steps θ can be approximated by the

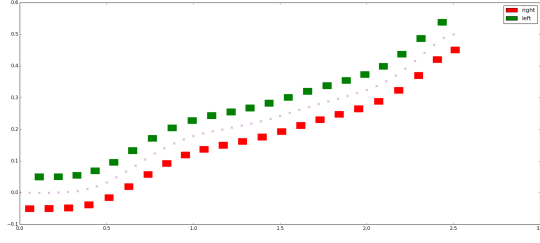


Fig. 6: Footsteps plan. With crosses stand for "principle steps", red- and green patches show the right and left foot, respectively.

following equation

$$\theta = \arctan \left(\frac{x_{i+1} - x_i}{y_{i+1} - y_i} \right). \quad (12)$$

With θ defined, we further set the deviation of the feet as d to ensure that the feet would not collide with each other, it is shown in the figure 5 with red dashed lines. We can then calculate the position of actual steps as

$$\begin{bmatrix} x_{\text{step},i} \\ y_{\text{step},i} \end{bmatrix} = \begin{bmatrix} (-1)^{i+1} \cdot d \cdot \cos \theta \\ (-1)^i \cdot d \cdot \sin \theta \end{bmatrix} + \begin{bmatrix} x_i \\ y_i \end{bmatrix}, \quad (13)$$

where $[x_i, y_i]^\top$ is the i -th position of principle step and $[x_{\text{step},i}, y_{\text{step},i}]^\top$ represents the actual position of corresponding step. We define the initial support foot to be "LLeg" (left leg index of NAO's convention), meaning that the robot always moves right foot as the first step. Given Bézier curve shown in figure 4, the calculated steps are interpreted in figure 6.

Finally, the steps shall be transformed into the support foot frame. This is intuitively done by recursively abstracting the coordinates of previous step from the current one. After all the steps are transformed, it is necessary to plan a final step to balance the robot to stand firmly with both feet aligned. For that, one can simply concatenate $[0, -d, 0]$ if the last planned step is left or $[0, d, 0]$ if it is the other.

IV. EVALUATION

The modules that required testing were the A* path planning, combined with the map creation and footstep planning after smoothing the trajectory using Bezier curves. The latter two have proven to work reliable in theory. The footstep planner has also shown satisfying results, proving a stable walking without any falling of the robot. The map creation and A* planning were tested before the hardware tests using example scenarios. The results are depicted in figures 7 and 8. Both modules are behaving as expected. Combined with the successful results from the Bezier curves and the footstep planner, the robot control has proven to be successful.

The last module that needed to be tested is the ArUco detection in the camera module and the Kalman. Both were tested in practice after the previous tests. Filter Videos of the final test results are saved in <https://1drv.ms/f/c/2ce7773a91366522/>

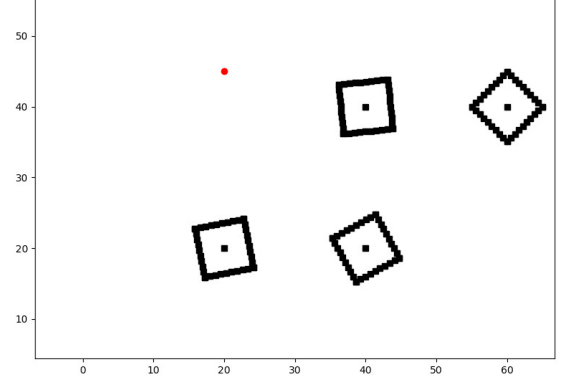


Fig. 7: Discretized map using obstacles of the same shape and size with different positions and orientations. The convex hull of the robot's feet was added to the obstacle dimensions to avoid potential collisions.

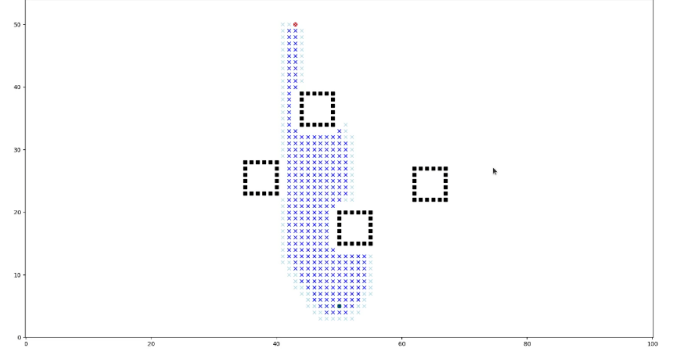


Fig. 8: A* and map creation algorithm using object positions and orientations in a discretized map

EqnJkhys7opNnb1zBviddyMBQLURKCxrU7GFk2dqIYJrOg. The robot is able to correctly determine the position and orientation of the detected ArUco markers as depicted in the rviz window. The robot is able to find trajectories to the goal and adapt accordingly to incoming obstacles. The fall recovery has not been needed in practice thus far, since the footstep planning has been proven to be robust to uncertainties. Modular tests in which this functionality has been tested in isolation have shown that the robot is able to reliably stand up on its own and resume walking towards the goal.

CONCLUSION

The NAO robot is able to achieve the desired goals. It is able to reach the goal in a reliable manner and adjust its trajectory accordingly once an obstacle is detected. The estimation through the Kalman filter adds robustness to the localization, enabling the robot to steer reliably through the map towards the goal. Thanks to this, the robot is able to execute the

footstep plan reliably. Thus far, the model has only been tested on static environments with deterministic outcomes. Future work would include adding further stability in path and footstep planning in environments with uncertainties. The robot would need to estimate the risk of every footstep on the floor and shall act accordingly to prevent falling.

REFERENCES

- [1] Senay Baydas and Bulent Karakas. Defining a curve as a bezier curve. *Journal of Taibah University for Science*, 13(1):522–528, 2019.
- [2] doxygen. Detection of aruco markers, 2016. https://docs.opencv.org/3.2.0/d5/dae/tutorial_aruco_detection.html.
- [3] GeeksforGeeks. A* search algorithm, 2024. <https://www.geeksforgeeks.org/a-search-algorithm/>.
- [4] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.
- [5] Oleg Kalachev. Aruco markers generator! <https://chev.me/arucogen/>.
- [6] EC Levy. Complex-curve fitting. *IRE transactions on automatic control*, (1):37–43, 1959.
- [7] Dan Simon. Kalman filtering. *Embedded systems programming*, 14(6):72–79, 2001.